

Jess Dowd

CS 499: Computer Science Capstone

Dr. Sherri Maciosek

November 29, 2025

5-2 Milestone Four: Enhancement Three: Databases Narrative

For my third enhancement I have improved the Travlr gateways website artifact again. I built this project for CS 465 in 2025. It uses MEAN stack, which is (Mongo, Express, Angular, Node). It shows trips and info for vacationing, and allows editing of the trips. It has a RESTful API that allows CRUD operations through HTTP methods to handle data requests between the client and server.

For this enhancement I have added and improved several features with clean, easily readable and secure code. I have expanded the database functionality by creating a MongoDB administrative interface using Angular components and templates built with TypeScript, HTML, and CSS to display and manage user data. The new admin account page lists all registered users, their roles, and their account details. The page is only be viewable by accounts with an admin role type.

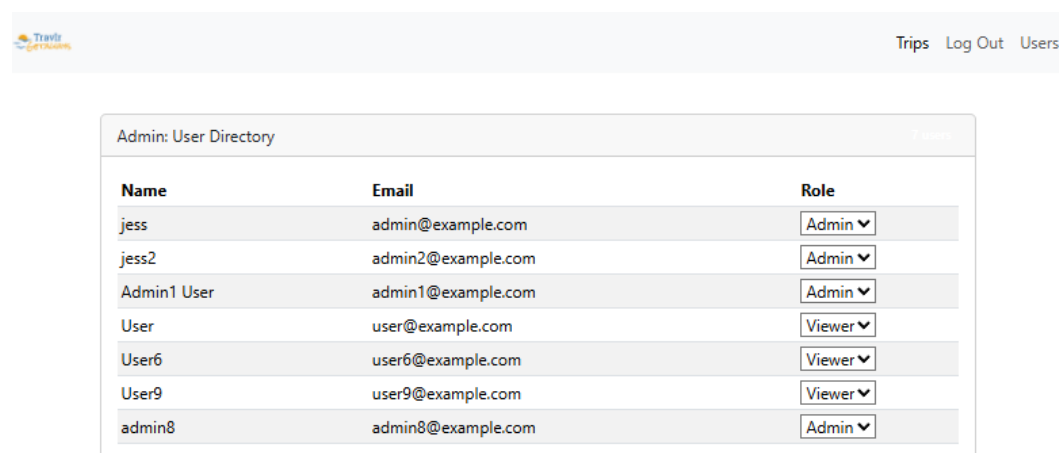
I selected this artifact for enhancement because I want to demonstrate my skills in web development within a MEAN full stack environment by creating an interactive admin interface that visualizes and manages data stored in MongoDB. This will help highlight my skills in advanced CRUD operations and MongoDB query operators to set filters and return data. Also

creating a database view page shows that I have the skills to use integrate backend database logic with a user interface, while using up to date best practices in the industry. It will also show my skills in secure data handling such as restricting access with JWT and hashing and salting sensitive fields when needed, and creating angular admin tools stack. This fits the database category because the enhancement creates new endpoints, new database queries, new data-access logic, and a full MongoDB-driven admin interface that did not exist before. This aligns with several of the course outcomes and also aligns with my career goals.

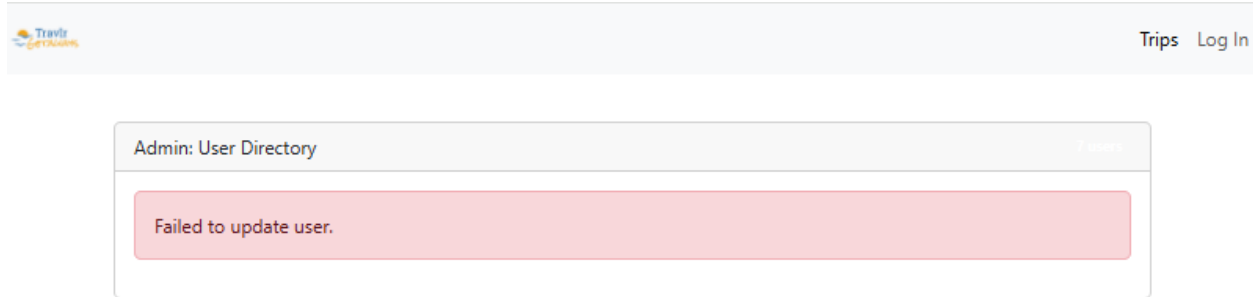
To implement this enhancement, I built new Angular front-end components that show an Admin “Users” page sends requests to the backend and displays a formatted table of accounts. I created new Express routes like GET /api/users and PATCH /api/users/:id that query or update user data, so that the admin can use the new page to update account info.

I coded it so the Express controller will only allow edits to the type field, by validating the request body and sanitizing input before performing database updates, which further enhances security and allows more flexibility with future enhancements.

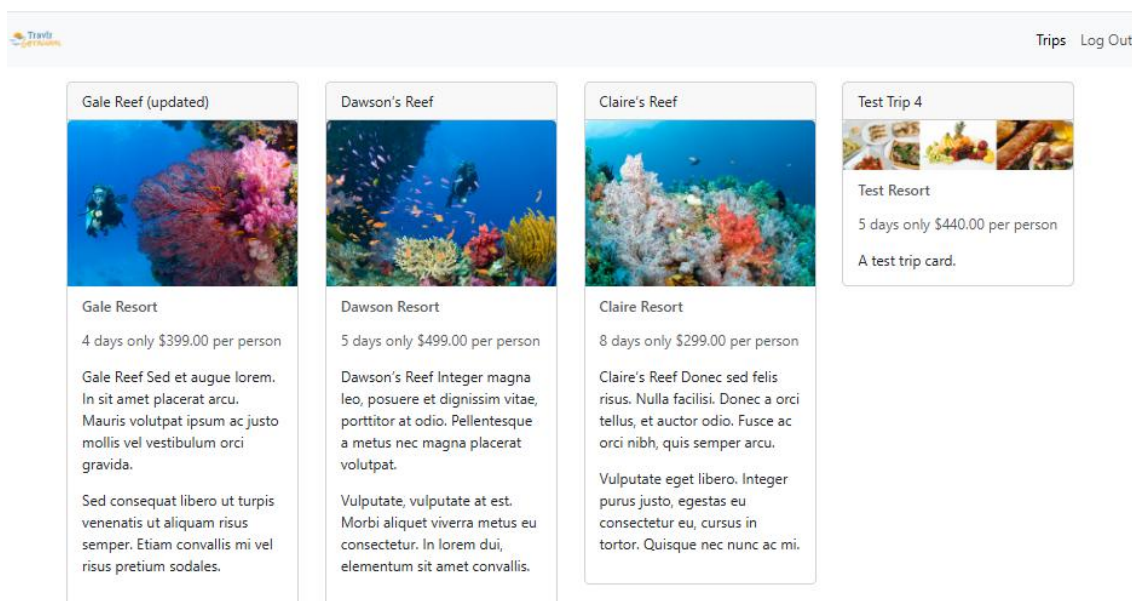
The photo below shows the view when logged in as admin of the user directory page. Includes dropdown to edit user account types. In the upper right corner, the user is able to navigate between different pages to edit the trips or user account info.



Error screen if the account info fails to update for any reason:



The viewer account type when logged in can't access or see the user page:



Created 2 user admin endpoints in the backend

Since I already have JWT auth working and RBAC middleware installed from the previous enhancement, I created a controller file called `users.js` with `listUsers` and `updateUser`. `listUsers` runs `find()` with a projection `'name email role'` so it only returns safe fields and not the

password hash or salt. Also both endpoints use `authenticateJWT` and `requireRole('admin')` which blocks viewer accounts.

```
// GET /api/users (admin only)
const listUsers = async (req, res) => {
```

```
const updateUser = async (req, res) => {
  try {
    // Build an object of allowed fields only.
    // Enhancement 3: Prevents updates to unsafe fields like password/hash.
    const allowed = {};
    if (req.body.name) allowed.name = req.body.name.trim();
    if (req.body.email) allowed.email = req.body.email.trim().toLowerCase();
    if (req.body.role) allowed.role = req.body.role;
```

Created admin only user management endpoints

I imported the new users file to `index.js`, and created 2 new api endpoints. Both routes use the `authenticateJWT` middleware which I built previously to validate the token and `requireRole('admin')` to block non-admin accounts, so only admins can access the user list or update data.

```
// ADMIN-ONLY USER MANAGEMENT (Enhancement 3)
// These routes were added to support the new admin user directory page.
// - GET lists all users
// - PUT updates allowed fields
// Both require a valid JWT and admin role.
// -----
router.get('/users', authenticateJWT, requireRole('admin'), ctrlUsers.listUsers);
router.put('/users/:userId', authenticateJWT, requireRole('admin'), ctrlUsers.updateUser);
```

Added user model to the frontend

The admin page will show the name, email, and role, and it needs the `_id` to send in PUT.

I added a TypeScript interface for user objects so Angular can type the data returned from the API.

```
// MongoDB document ID.  
// Enhancement 3: Required so the admin page can send PUT /api/users/:id.  
_id?: string;  
  
constructor() {  
  this.email = '';  
  this.name = '';  
  this.role = '';  
  this._id = '';  
}
```

Added the new API calls

I added two new methods to `TripDataService`. One to get all of the users and the other to update a user. These two service methods call the backend using Angular `HttpClient` and automatically attach the JWT token using the interceptor.

```
// Admin user management (Enhancement 3) -----  
  
// Get list of all users (admin only)  
// Enhancement 3: Added for the new admin user directory page.  
getAllUsers(): Observable<User[]> {  
  return this.http.get<User[]>(`${this.baseUrl}/users`);  
}  
  
// Update a user by ID (admin only)  
// Enhancement 3: Used when admin changes a user's role or info.  
updateUser(id: string, data: Partial<User>): Observable<User> {  
  return this.http.put<User>(`${this.baseUrl}/users/${id}`, data);  
}
```

Added an Admin Users Component on the frontend

I generated a new angular user component in the root directory using powershell, then added methods to my TripDataService.

```
PS C:\Users\User\CS-465\travlr\app_admin> ng g component users-admin
CREATE src/app/users-admin/users-admin.ts (215 bytes)
CREATE src/app/users-admin/users-admin.css (0 bytes)
CREATE src/app/users-admin/users-admin.html (27 bytes)
```

Users Admin TypeScript

I replaced the default contents that generated in the components with a new class called onRoleChange. This is the core logic for the new admin directory. On init, when the component loads, it checks auth.isAdmin() and stops if the user is not an admin, which prevents unauthorized access even on the frontend. If it is an admin it will then get all of the users from TripDataService and store the list of data in this.users. Then when the admin wants to make a edit to account info, onRoleChange sends a PUT to update the user's info.

```
// Called when admin selects a new role in the dropdown
// Enhancement 3: Sends PUT /api/users/:id to update user data.
onRoleChange(u: User, newRole: any): void {
  this.tripDataService.updateUser(u._id as string, { role: newRole }).subscribe({
    next: (updated) => {
      u.role = updated.role; // Update UI after successful save
    },
    error: () => {
      this.error = 'Failed to update user.';
    }
  });
}
```

Built the HTML table UI

I replaced the Angular template with custom HTML. It shows a table of users with their name, role, and email. It has a dropdown menu that allows the admin to change the user account role. The dropdown menu uses (ngModelChange) to detect updates and calls onRoleChange() which uses PUT with /api/users/:id

Admin: User Directory		
Name	Email	Role
jess	admin@example.com	Admin ▼
jess2	admin2@example.com	Admin ▼
Admin1 User	admin1@example.com	Admin ▼
User	user@example.com	Viewer ▼
User6	user6@example.com	Viewer ▼
User9	user9@example.com	Viewer ▼
admin8	admin8@example.com	Admin ▼
Admin Test	admin@test.dev	Admin ▼
Viewer Test	viewer@test.dev	Viewer ▼

Added the route to the users admin page

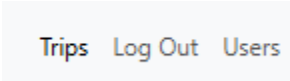
Angular uses standalone components so I had to import UsersAdminComponent into app.routes instead of using an NgModule. I imported UserAdminComponent into app.routes and added a new routes in my route array, that makes it so the new admin page will load on the browser at the correct url at <http://localhost:4200/admin/users>

```
import { UsersAdminComponent } from './users-admin/users-admin';
```

```
{ path: 'admin/users', component: UsersAdminComponent },
```

Added a link in the navbar

In the Navbar HTML I added a routerLink that links the button to admin/users page, and that checks if the user is an admin with auth.isAdmin, which hides the button if the user is a viewer account so that that only the admin can see. Viewer accounts are blocked on the frontend by hiding the UI and on the backend since RBAC middleware enforces the restriction.



Trips Log Out Users

```
<!-- Enhancement 3:  
| | | New admin-only link to the user directory page -->  
<a class="nav-link" *ngIf="auth.isAdmin()" routerLink="/admin/users">Users</a>
```

I met all five course outcomes with this enhancement. The first course outcome is met since this enhancement enables collaborative environments using data management because it lets admins monitor and manage user information through the database interface, which allows better and easier decision making when all of the data is easily viewable.

I met the second course outcome by developing an interactive admin interface that demonstrates my skills in visual communication of complex data because I created a clear and easy to read account page that will be accessible to website admins.

I met the third course outcome because I used algorithmic principles and computer science best practices to solve the issue of needing a way to display and edit account info. I evaluated design choices and tradeoffs for this new feature given the scope and time constraints, and maximized the feature in the areas that best focuses with my career and the skills I want to show employers.

I met the fourth course outcome by Integrating MongoDB queries and a dynamic front-end display and showing real-world database application in the MEAN stack which demonstrates my ability to use innovative computing practices to help my industry specific goals in web development.

I met the fifth course outcome because my enhancement promotes accountability and secure data handling which shows I have a security mindset able to anticipate exploits and mitigate those potential issues. The user management system reinforces secure database design by controlling data access from unauthorized users and allowing updates to sensitive info by authorized users.

While enhancing and modifying this artifact I improved my skills working with advanced CRUD operations and query filters. Also creating a database view page improved my skills to use build database logic along with a user interface, and security and data best practices. It also helped me improve my skills in secure data handling, and creating admin tools in MEAN stack. This aligns with several of the course outcomes and also aligns with my career goals.

I encountered some challenges throughout the enhancement including managing time to choose which aspects and in which order to implement the enhancement steps. I ran into some bugs which presented a challenge but I was able to solve. For example, at one point the server was no longer able to load after a change I had made, I was able to go into the console and read the error output to discover that I had mistakenly set my Authentication service to private, which blocked Angular from accessing I during the admin check.

```
x [ERROR] TS2341: Property 'auth' is private and only accessible within class 'NavbarComponent'. [plugin angular-compiler]
src/app/navbar/navbar.html:14:14:
14 | <a *ngIf="auth.isAdmin()" routerLink="/admin/users">Users</a>
```

```
export class NavbarComponent implements OnInit {  
  constructor(public auth: AuthenticationService) {}  
  ngOnInit(): void {}  
}
```

Also, I also had originally forgotten to import the CommonModule, and FormsModule to my new users typescript file because each component is standalone and needs to be connected to be used.

```
▲ WARNING NG8103: The `*ngIf` directive was used in the template, but neither the `NgIf` directive nor the `CommonModule` was imported. Use Angular's built-in control flow @if or make sure that either the `NgIf` directive or the `CommonModule` is included in the `@Component.imports` array of this component. [plugin angular-compiler]
```